

Guida Didattica: Meta-Analisi mTBI — Versione Python v10 Publication Ready

Per studenti delle scuole superiori

Indice

1. Che Cosa Stiamo Facendo (in Parole Semplici)
 2. Le Basi: Come Si Valuta un Test Medico
 3. Perché Combinare Più Studi? La Meta-Analisi
 4. Il Modello Statistico: DerSimonian-Laird Spiegato Facile
 5. Differenze tra R e Python: Cosa è Cambiato
 6. Python Può Introdurre Bias? La Verità
 7. Spiegazione del Codice Python — Riga per Riga
 8. Perché Python per le Figure “Publication Ready”
 9. Riassunto Finale
-

1. Che Cosa Stiamo Facendo (in Parole Semplici)

Il Problema Reale

Immagina di essere al Pronto Soccorso alle 2 del mattino. Arriva **Luca, 28 anni**, dopo essere caduto dallo skateboard. Ha battuto la testa, è stato un po’ confuso per qualche minuto, ora sta bene.

Il medico si chiede: “Devo fare la TAC cranica a Luca?”

- **Se faccio la TAC a tutti** → molti pazienti ricevono radiazioni inutilmente
- **Se non faccio la TAC a nessuno** → rischio di mancare emorragie pericolose

La soluzione: usare un **esame del sangue** (biomarcatore) che ci dice se è probabile che ci sia una lesione nel cervello.

Cosa Vogliamo Ottenere

Esistono **due biomarcatori** principali:

Biomarcatore	Quanti studi lo hanno valutato?	Quanti pazienti totali?
GFAP + UCH-L1	8 studi	4.550
S100B	6 studi	3.582

Ogni studio ha dato risultati leggermente diversi. Il nostro obiettivo è **combinare tutti questi risultati** per rispondere a due domande:

1. **Quanto è affidabile** ciascun biomarcatore?
2. **Quale dei due è migliore** per decidere chi fare la TAC?

Come Lo Facciamo

Usiamo una tecnica statistica chiamata **meta-analisi**, che è come fare una “ricetta” che mescola i risultati di tutti gli studi per ottenere un risultato più preciso e affidabile.

2. Le Basi: Come Si Valuta un Test Medico

La Tabella 2×2: Il Fondamento di Tutto

Ogni studio che valuta un test medico produce quattro numeri. Ecco come si ottengono:

Prendiamo 1.000 pazienti arrivati al PS con un trauma cranico lieve. A tutti viene fatto:
- **Il test del sangue** (biomarcatore: positivo o negativo) - **La TAC** (il “gold standard”: lesione presente o assente)

Poi si incrociano i risultati:

	TAC positiva (lesione C'È)	TAC negativa (lesione NON c'è)
Test positivo	TP = 50 (Veri Positivi)	FP = 300 (Falsi Positivi)
Test negativo	FN = 5 (Falsi Negativi)	TN = 645 (Veri Negativi)

I Due Numeri Più Importanti

1. Sensibilità: “Quanto è bravo il test a TROVARE le lesioni?”

$$\text{Sensibilità} = TP / (TP + FN) = 50 / (50 + 5) = 50/55 = 90.9\%$$

Traduzione: Su 100 pazienti che hanno DAVVERO una lesione, il test ne trova 91. Ne “perde” 9.

Più alta è la sensibilità, meglio è. Una sensibilità del 100% significa che il test non si perde mai una lesione.

2. Specificità: “Quanto è bravo il test a ESCLUDERE chi sta bene?”

$$\text{Specificità} = TN / (TN + FP) = 645 / (645 + 300) = 645/945 = 68.3\%$$

Traduzione: Su 100 pazienti che NON hanno una lesione, il test dice correttamente che 68 stanno bene. Ma 32 vengono sbagliati (risultano positivi quando stanno bene).

Più alta è la specificità, meglio è. Una specificità del 100% significa che il test non sbaglia mai su chi sta bene.

Il Compromesso (Trade-off)

Qui c'è il punto cruciale: **sensibilità e specificità sono nemiche.**

- Se vuoi che il test trovi TUTTE le lesioni (sensibilità alta) → devi abbassare la soglia di positività → ma così più persone sane risultano positive (specificità bassa)
- Se vuoi che il test non sbagli sui sani (specificità alta) → devi alzare la soglia → ma così ti perdi qualche lesione (sensibilità bassa)

È come un metal detector in aeroporto: - Se lo tarassi super-sensibile → suona per ogni moneta, ma non ti perdi un'arma (sensibilità alta, specificità bassa) - Se lo tarassi poco sensibile → non ti perdi un'arma grossa, ma fai passare molte persone senza controlli (specificità alta, sensibilità bassa)

Il Terzo Numero: NPV (Valore Predittivo Negativo)

Questo è il numero che interessa davvero al medico:

NPV = "Se il test è negativo, quanto posso fidarmi?"

Se il NPV è 99%, significa: **su 100 pazienti con test negativo, 99 stanno davvero bene.**

Per il trauma cranico lieve, il NPV è il numero più importante, perché la domanda del medico è: "Posso mandare questo paziente a casa senza TAC?"

3. Perché Combinare Più Studi? La Meta-Analisi

Il Problema dei Singoli Studi

Ogni studio ha un problema: **il campione è piccolo.**

Esempio: lo studio "Jones 2020" ha valutato il GFAP su 679 pazienti. Sembra tanto, ma: - Solo 33 avevano una lesione (CT+) - Se per caso in quel campione c'erano pazienti un po' diversi, il risultato cambia

È come fare un sondaggio elettorale intervistando solo 50 persone: il risultato potrebbe essere molto diverso dalla realtà.

La Soluzione: La Meta-Analisi

La meta-analisi **combina** tutti gli studi insieme, come se fossero un unico gigantesco studio.

Analogia culinaria: - Ogni studio è un assaggio di minestra da un cucchiaino diverso - Un singolo assaggio potrebbe non essere rappresentativo (magari quel cucchiaino ha preso solo patate) - La meta-analisi è come mescolare tutta la minestra e assaggiare il risultato complessivo

I Vantaggi

Singolo studio	Meta-analisi
Campione piccolo	Campione enorme (migliaia di pazienti)
Risultato impreciso	Risultato preciso
Potrebbe essere “fortunato” o “sfortunato”	I casi si bilanciano
Vale per quella popolazione	Vale in generale

4. Il Modello Statistico: DerSimonian-Laird Spiegato Facile

Il Problema: Gli Studi Sono Diversi tra Loro

I nostri 8 studi sul GFAP non danno tutti lo stesso risultato:

Studio	Sensibilità
Oris 2024	100%
Bazarian 2021	96%
Papa 2022	98%
...	...

Perché sono diversi? Per due motivi:

1. **Caso (errore campionario):** Ogni studio ha un po' di variabilità casuale
2. **Differenze vere:** I pazienti erano diversi (età diversa, gravità diversa, ospedale diverso)

La Soluzione: Il Modello ad Effetti Random

Il modello **DerSimonian-Laird** (il nome viene dai due statistici che l'hanno inventato nel 1986) funziona così:

Passo 1: Trasformare i Dati Le sensibilità sono numeri tra 0 e 1 (es. 0.95). Ma per fare calcoli statistici è più comodo lavorare con numeri che vanno da meno infinito a più infinito.

Si usa una trasformazione chiamata **logit**:

$$\text{logit}(0.95) = \ln(0.95 / 0.05) = \ln(19) \quad 2.94$$

Perché? Perché dopo possiamo usare la “campana” di Gauss (la distribuzione normale), che funziona su tutta la retta dei numeri.

Passo 2: Dare un Peso a Ogni Studio Non tutti gli studi contano uguale. **Gli studi più grandi e più precisi contano di più.**

$$\text{Peso} = 1 / \text{varianza}$$

- Studio grande (es. 1000 pazienti) → varianza piccola → peso grande → conta molto
- Studio piccolo (es. 100 pazienti) → varianza grande → peso piccolo → conta meno

È come un voto di maturità: l'esame di stato vale più di una verifica in classe.

Passo 3: Misurare Quanto gli Studi Sono Diversi (Eterogeneità) Si calcola un numero chiamato **Q (statistica di Cochran)**:

$$Q = \sum [\text{peso}_i \times (\text{risultato}_i - \text{media})^2]$$

- Se Q è piccolo → gli studi sono tutti simili tra loro (basta il caso a spiegare le differenze)
- Se Q è grande → gli studi sono davvero diversi (c'è qualcosa di più del caso)

Da Q si ricava τ^2 (**tau quadro**), che misura quanta variabilità “vera” c'è tra gli studi.

Passo 4: Calcolare la Media Ponderata “Random-Effects” Ora si ricalcola la media, ma questa volta i pesi includono anche τ^2 :

$$\text{Nuovo peso} = 1 / (\text{varianza} + \tau^2)$$

$$\text{Media pooled} = \sum (\text{nuovo_peso}_i \times \text{risultato}_i) / \sum \text{nuovo_peso}_i$$

Cosa cambia rispetto a una media semplice? - Gli studi molto grandi perdono un po' di peso (perché τ^2 aggiunge incertezza) - Il risultato finale è più “cauto” e realistico

Passo 5: Ritrasformare il Risultato La media pooled è in scala logit. Si ritrasforma con la **expit** (l'inversa del logit):

$$\text{expit}(x) = 1 / (1 + e^{(-x)})$$

E voilà: otteniamo la sensibilità pooled, un numero tra 0 e 1 (es. 0.93 = 93%).

L'Intervallo di Confidenza al 95%

Non basta dire “la sensibilità è 93%”. Dobbiamo dire **quanto siamo sicuri** di questo numero.

L'intervallo di confidenza al 95% ci dice:

Sensibilità = 93% [90% - 96%]

Cosa significa: siamo ragionevolmente sicuri che la vera sensibilità nella popolazione sia tra il 90% e il 96%.

Analogia: è come dire “la temperatura domani sarà 25°C, più o meno 2-3 gradi”. Non è una certezza assoluta, ma è un’informazione utile.

I²: Quanto Conta la Variabilità Vera?

I² = percentuale della variabilità totale che NON è dovuta al caso

I ²	Significato
0-25%	Eterogeneità bassa (gli studi sono molto simili)
25-50%	Eterogeneità moderata
50-75%	Eterogeneità alta
>75%	Eterogeneità molto alta (gli studi sono molto diversi)

5. Differenze tra R e Python: Cosa è Cambiato

Il Modello Statistico: È lo Stesso?

Sì e no. Ecco le differenze:

Aspetto	Versione R (v9)	Versione Python (v10)
Modello	Reitsma bivariato (maximum likelihood)	DerSimonian-Laird su logit(Se) e logit(Sp) separatamente
Correlazione Se-Sp	Modellata esplicitamente	Non modellata (marginale)
Stima di τ^2	Maximum likelihood (REML)	DerSimonian-Laird (metodo dei momenti)
Risultati	Leggermente diversi	Leggermente diversi, ma molto simili

Spiegazione Semplice

R (pacchetto mada): - Usa il modello di Reitsma “vero”: tratta sensibilità e specificità **insieme**, come una coppia correlata - È come misurare altezza e peso di una persona sapendo che sono collegati (le persone alte tendono a pesare di più) - Più sofisticato, più preciso

Python (v10): - Usa DerSimonian-Laird: tratta sensibilità e specificità **separatamente** - È come misurare altezza e peso indipendentemente, senza considerare che sono collegati - Più semplice, più trasparente, ma leggermente meno preciso

È come due modi di fare la stessa ricetta: - R usa il robot da cucina professionale (massima verosimiglianza) - Python usa le mani e un cucchiaino di legno (metodo dei momenti) - Il risultato è simile, ma non identico

Cosa Cambia nei Numeri?

Le differenze sono piccole (1-2 punti percentuali al massimo). Per esempio:

	R (Reitsma)	Python (DL)
GFAP Sensibilità	~93%	~92-93%
GFAP Specificità	~40%	~38-40%

Perché Due Versioni?

1. **R** è stato usato per l'analisi iniziale e per capire i dati
2. **Python** è stato usato per produrre le **figure publication-ready** (vedi sotto)

6. Python Può Introdurre Bias? La Verità

La Domanda Giusta

“Usare Python invece di R mi sta dando risultati sbagliati?”

La Risposta Onesta

Ci sono differenze, ma non sono “bias” nel senso grave del termine. Ecco cosa succede:

Cosa Python fa CORRETTAMENTE

Aspetto	Stato
Trasformazione logit	Corretta
Correzione per celle a zero (+0.5)	Applicata
Pesi inversi della varianza	Corretti
Formula DerSimonian-Laird	Implementata correttamente
Intervalli di confidenza	Calcolati correttamente
Wilson score CI per singoli studi	Corretto

Cosa è DIVERSO (ma non “sbagliato”)

Aspetto	Impatto
Nessuna correlazione Se-Sp	Le stime pooled sono leggermente diverse (1-2%). Non è un errore, è un modello diverso.
DerSimonian-Laird vs REML	DL tende a sottostimare τ^2 quando ci sono pochi studi (<10). Con 6-8 studi, l'effetto è piccolo.
Curve SROC approssimate	Il metodo Moses-Shapiro-Littenberg è un'approssimazione. Le curve sono indicative, non stime formali.

Cosa POTREBBE Essere Problematico

Problema	Rischio	Come Mitigare
Pochi studi ($k < 10$) + DL	τ^2 potrebbe essere sottostimato $\rightarrow$ IC troppo stretti	Usare REML o Bayesian se possibile
Zero cells	La correzione +0.5 è arbitraria	Fare sensitivity analysis con correzioni diverse
Eterogeneità alta ($I^2 > 75\%$)	La media pooled potrebbe non avere senso	Esplorare le fonti di eterogeneità

Il Verdetto

Python NON introduce bias sistematici gravi. Le differenze con R sono dovute al modello statistico diverso (DL vs Reitsma), non a errori di programmazione. Per una pubblicazione scientifica, è buona pratica:

1. **Dichiarare** quale metodo si è usato
2. **Confrontare** i risultati con entrambi i metodi (se possibile)
3. **Essere trasparenti** sui limiti (pochi studi, eterogeneità)

Analogia

È come misurare la temperatura con due termometri diversi: - Termometro A (R/Reitsma): termometro digitale di precisione - Termometro B (Python/DL): termometro a mercurio classico

Entrambi ti dicono se hai la febbre. La differenza è di qualche decimo di grado. Per la diagnosi va bene entrambi. Per la ricerca di precisione, meglio il digitale.

7. Spiegazione del Codice Python — Riga per Riga

PARTE 1: Preparazione

```
#!/usr/bin/env python3
```

Cosa fa: Dice al sistema operativo di eseguire questo file con Python 3. È come l'intestazione di una lettera.

```
"""
mTBI Biomarker DTA Meta-Analysis - Version 10 Publication Ready
Produce:
    1. Console output identico a R summary(reitsma)
    2. Fig2 - Forest plot GFAP+UCH-L1 (publication ready, NEJM/Lancet style)
    3. Fig3 - Forest plot S100B
    4. Fig4 - SROC comparativo
    5. Fig5 - Clinical summary panel (4 subplots)
    6. Table1.png - Summary table publication ready
"""
```

Cosa fa: È un commento che spiega cosa fa il file. I tre virgolette """ indicano un testo su più righe.

PARTE 2: Import delle Librerie

```
import numpy as np
```

Cosa fa: Importa NumPy, la libreria per fare calcoli matematici con vettori e matrici. `as np` è un'abbreviazione: invece di scrivere `numpy.array()` scrivo `np.array()`.

```
import matplotlib
matplotlib.use('Agg')
```

Cosa fa: Configura matplotlib (la libreria per i grafici) per funzionare **senza schermo**. 'Agg' significa che salva i file direttamente senza aprire finestre. Utile quando il codice gira su un server.

```
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import matplotlib.gridspec as gridspec
from matplotlib.patches import FancyBboxPatch, Polygon
from matplotlib.lines import Line2D
```

Cosa fa: Importa vari strumenti per disegnare: - `pyplot`: il modulo principale per creare grafici - `patches`: forme geometriche (rombi, rettangoli) - `gridspec`: per organizzare più grafici in una pagina - `Polygon`: per disegnare il rombo pooled nei forest plot - `Line2D`: per le linee nella legenda

```
from scipy.special import expit, logit
from scipy.stats import chi2, norm
```

Cosa fa: Importa funzioni matematiche speciali: - **expit**: la funzione logistica (inversa del logit): $\text{expit}(x) = 1/(1+e^{(-x)})$ - **logit**: la trasformazione logit: $\text{logit}(p) = \ln(p/(1-p))$ - **chi2**: la distribuzione chi-quadro (serve per il test di eterogeneità) - **norm**: la distribuzione normale (campana di Gauss)

```
import warnings
warnings.filterwarnings('ignore')
```

Cosa fa: Dice a Python di **non mostrare gli avvisi**. A volte Python si lamenta (es. “attenzione, stai dividendo per zero”), ma in questi casi sappiamo cosa stiamo facendo e gli avvisi non servono.

PARTE 3: Stile Grafico da Rivista Scientifica

```
plt.rcParams.update({
    'font.family':      'DejaVu Sans',
    'axes.spines.top':   False,
    'axes.spines.right': False,
    'axes.linewidth':    0.8,
    'xtick.major.width': 0.8,
    'ytick.major.width': 0.8,
    'xtick.labelsize':   9,
    'ytick.labelsize':   9,
    'axes.labelsize':    10,
    'axes.titlesize':    11,
    'legend.fontsize':   9,
    'figure.dpi':        300,
})
```

Cosa fa: Imposta lo stile grafico per sembrare un articolo su **NEJM o Lancet** (le riviste mediche più prestigiose al mondo).

Ogni riga cambia un'impostazione:

Impostazione	Cosa Cambia
<code>font.family</code>	Tipo di carattere: DejaVu Sans (pulito, leggibile)
<code>axes.spines.top/right</code>	Niente linee sopra e a destra del grafico (stile moderno)
<code>axes.linewidth</code>	Linee degli assi sottili (0.8 pixel)
<code>xtick.labelsize</code>	Numeri sugli assi piccoli (9 punti)

Impostazione	Cosa Cambia
<code>figure.dpi</code>	Risoluzione alta: 300 punti per pollice (qualità stampa)

```
COL_GFAP    = '#1B4F8A'    # deep blue
COL_S100B   = '#1A6B3C'    # deep green
COL_GFAP_L  = '#90B8D8'    # light blue
COL_S100B_L = '#90C9A8'    # light green
COL_GREY    = '#4A4A4A'
COL_LGREY   = '#E8E8E8'
COL_BLACK   = '#1A1A1A'
```

Cosa fa: Definisce i colori usando codici esadecimali (come il web).

- GFAP = blu scuro (colore “serio”, professionale)
- S100B = verde scuro (per distinguerlo chiaramente)
- Le versioni _L (light) sono più chiare, usate per gli sfondi

Perché colori specifici? Le riviste scientifiche vogliono colori che: 1. Si distinguano bene anche in bianco e nero 2. Siano accessibili a chi ha daltonismo 3. Siano coerenti in tutte le figure

PARTE 4: I Dati

```
GFAP_RAW = dict(
    study = ["Oris 2024",
             "Bazarian 2021",
             "Papa 2022",
             "Chayoua 2024",
             "Legramante 2024",
             "Jones 2020",
             "Lagares 2024\n(BRAINI)",
             "Puravet 2025"],
    ref    = ["CCLM 2024", "AEM 2021", "JAMA NO 2022", "JNT 2024",
             "IJEM 2024", "Brain Inj 2020", "EBioMed 2024", "AEM 2025"],
    TP     = np.array([18, 24, 52, 38, 7, 33, 68, 99]),
    FP     = np.array([61, 199, 392, 130, 89, 425, 477, 285]),
    FN     = np.array([0, 1, 1, 1, 0, 0, 2, 1]),
    TN     = np.array([127, 167, 413, 149, 34, 221, 663, 373]),
)
```

Cosa fa: Crea un dizionario con i dati degli 8 studi sul GFAP.

- `study`: nomi degli studi (autore + anno)

- **ref**: la rivista dove sono stati pubblicati
- **TP, FP, FN, TN**: i 4 numeri della tabella 2×2 per ogni studio
- **np.array(...)**: crea un vettore di numeri (più efficiente di una lista)

Nota: `\n` nel nome “Lagares 2024” significa “a capo”. Serve per far stare il nome nel grafico su due righe.

Lo stesso blocco viene ripetuto per S100B con S100B_RAW.

PARTE 5: La Funzione `enrich()`

```
def enrich(d, add=0.5):
    d = dict(d)
    tp = d['TP'].astype(float); fp = d['FP'].astype(float)
    fn = d['FN'].astype(float); tn = d['TN'].astype(float)
```

Cosa fa: Definisce una funzione che prende un dizionario di dati e calcola tutte le variabili derivate.

- **d = dict(d)**: crea una copia del dizionario (così non modifico l'originale)
- **.astype(float)**: converte i numeri interi in decimali (serve per le divisioni)

```
tp[tp==0]+=add; fn[fn==0]+=add; fp[fp==0]+=add; tn[tn==0]+=add
```

Cosa fa: Aggiunge 0.5 a tutte le celle che valgono zero.

Perché? Perché se $TP = 0$, allora $\text{logit}(TP/(TP+FN)) = \text{logit}(0) = -\infty$, che è un problema. Aggiungere 0.5 è una correzione standard chiamata **correzione di Haldane-Anscombe**.

Esempio pratico: - Studio “Oris 2024” ha $FN = 0$ (nessun falso negativo) - Senza correzione: $Se = 18/(18+0) = 1.0 \rightarrow \text{logit}(1.0) = +\infty \rightarrow \text{ERRORE}$ - Con correzione: $Se = 18.5/(18.5+0.5) = 0.974 \rightarrow \text{logit}(0.974) = 3.66 \rightarrow \text{OK}$

```
d.update(tp=tp,fp=fp,fn=fn,tn=tn)
d['N']    = d['TP']+d['FP']+d['FN']+d['TN']
d['Se']    = tp/(tp+fn)
d['Sp']    = tn/(tn+fp)
d['CTp']   = d['TP']+d['FN']
d['prev']  = d['CTp']/d['N']
```

Cosa fa: Calcola le variabili derivate: - **N**: totale pazienti per studio - **Se**: sensibilità per studio - **Sp**: specificità per studio - **CTp**: pazienti con CT positiva per studio - **prev**: prevalenza di CT+ per studio

```
d['lSe'] = np.log(d['Se']/(1-d['Se']))
d['lSp'] = np.log(d['Sp']/(1-d['Sp']))
d['vSe'] = 1/tp + 1/fn
```

```
d['vSp'] = 1/tn + 1/fp
return d
```

Cosa fa: Calcola le trasformate logit e le loro varianze.

- 1Se: logit della sensibilità
- 1Sp: logit della specificità
- vSe: varianza del logit(Se) = $1/TP + 1/FN$
- vSp: varianza del logit(Sp) = $1/TN + 1/FP$

Perché queste varianze? Derivano dalla teoria statistica: la varianza approssimata del logit di una proporzione p su n osservazioni è circa $1/(np) + 1/(n(1-p))$, che nel nostro caso diventa $1/TP + 1/FN$.

PARTE 6: La Funzione DL() — Il Cuore del Modello

```
def DL(y, v):
    k = len(y)
    w = 1/v
    mFE = np.sum(w*y)/np.sum(w)
```

Cosa fa: Definisce la funzione DerSimonian-Laird.

- y: i valori da combinare (es. i logit delle sensibilità)
- v: le varianze di questi valori
- k: numero di studi
- w: pesi inversi della varianza (studio più preciso = più peso)
- mFE: media ad effetti fissi (Fixed Effects)

La media ad effetti fissi è come una media pesata:

```
mFE = (w*y + w*y + ... + w*y) / (w + w + ... + w)

Q = np.sum(w*(y-mFE)**2)
C = np.sum(w)-np.sum(w**2)/np.sum(w)
t2 = max(0, (Q-(k-1))/C)
```

Cosa fa: Calcola l'eterogeneità.

- Q: statistica di Cochran. Se gli studi sono tutti uguali, $Q \approx k-1$. Se sono diversi, $Q > k-1$.
- C: una costante di correzione
- t^2 (t^2): la varianza inter-studio

La formula di t^2 :

```
 $t^2 = \max(0, (Q - (k-1)) / C)$ 
```

Il `max(0, ...)` serve perché τ^2 non può essere negativo. Se $Q < k-1$ (gli studi sono più simili del previsto per caso), $\tau^2 = 0$.

```
wRE = 1/(v+t2)
mu = np.sum(wRE*y)/np.sum(wRE)
se = np.sqrt(1/np.sum(wRE))
```

Cosa fa: Calcola la media random-effects.

- `wRE`: i nuovi pesi, che includono τ^2 (varianza intra + inter-studio)
- `mu`: la media pooled in scala logit
- `se`: l'errore standard della media

La differenza con la media ad effetti fissi: - Effetti fissi: $w = 1/\text{varianza}$ (solo errore campionario) - Random-effects: $wRE = 1/(\text{varianza} + \tau^2)$ (errore campionario + eterogeneità)

```
I2 = max(0, (Q-(k-1))/Q)*100 if Q>0 else 0.0
pQ = 1-chi2.cdf(Q, df=k-1)
```

Cosa fa: Calcola le misure di eterogeneità.

- I^2 : percentuale di variabilità dovuta a differenze vere (non caso)
- `pQ`: p-value del test di Cochran (se $p < 0.05$, c'è eterogeneità significativa)

```
return dict(est=expit(mu), lo=expit(mu-1.96*se), hi=expit(mu+1.96*se),
            tau2=t2, I2=I2, Q=Q, df=k-1, pQ=pQ, mu=mu, se_mu=se)
```

Cosa fa: Restituisce tutti i risultati.

- `est`: la stima pooled ritrasformata (tra 0 e 1)
- `lo`, `hi`: gli estremi dell'intervallo di confidenza al 95%
- `tau2`, `I2`, `Q`, `df`, `pQ`: misure di eterogeneità
- `mu`, `se_mu`: la media e il suo errore standard in scala logit

PARTE 7: Wilson Score, NPV, e Analisi

```
def wilson(p, n, z=1.96):
    c = (p+z**2/(2*n))/(1+z**2/n)
    m = z*np.sqrt(p*(1-p)/n+z**2/(4*n**2))/(1+z**2/n)
    return np.maximum(0, c-m), np.minimum(1, c+m)
```

Cosa fa: Calcola l'intervallo di confidenza con il metodo **Wilson score**.

Perché non il metodo semplice? Il metodo classico $p \pm 1.96 \times \sqrt{p(1-p)/n}$ funziona male quando: - p è vicino a 0 o 1 (come le sensibilità, spesso $>90\%$) - n è piccolo

Wilson score è più preciso in questi casi.

```
def npv_b(se, sp, prev):
    return (sp*(1-prev))/(sp*(1-prev)+(1-se)*prev)
```

Cosa fa: Calcola il NPV usando il teorema di Bayes.

Spiegazione: - Numeratore: probabilità di essere CT– E test negativo - Denominatore: probabilità totale di test negativo - Risultato: $P(\text{CT-} \mid \text{test negativo})$

```
def analyse(raw, label):
    d = enrich(raw)
    k = len(d['study'])
    Nt = d['N'].sum(); CTp = d['CTp'].sum()
    prev = CTp/Nt
    rSe = DL(d['lSe'], d['vSe'])
    rSp = DL(d['lSp'], d['vSp'])
    npv = npv_b(rSe['est'], rSp['est'], prev)
    ct = rSp['est']*(1-prev)*100
    miss = (1-rSe['est'])*prev*1000
    return dict(d=d, k=k, N=Nt, CTp=CTp, prev=prev,
               Se=rSe, Sp=rSp, npv=npv, ct=ct, miss=miss, label=label)
```

Cosa fa: Funzione “contenitore” che fa tutta l’analisi per un biomarcatore.

1. `enrich(raw)`: calcola tutte le variabili derivate
2. `DL(d['lSe'], d['vSe'])`: stima pooled della sensibilità
3. `DL(d['lSp'], d['vSp'])`: stima pooled della specificità
4. `npv_b(...)`: calcola il NPV
5. `ct`: % di TAC evitate
6. `miss`: lesioni mancate per 1000 pazienti

```
G = analyse(GFAP_RAW, "GFAP+UCH-L1")
S = analyse(S100B_RAW, "S100B")
```

Cosa fa: Esegue l’analisi per entrambi i biomarcatori. `G` = risultati GFAP, `S` = risultati S100B.

PARTE 8: Output Console (Stile R)

```
def print_summary(res, biomarker_label):
    d = res['d']
    Se = res['Se']; Sp = res['Sp']
    sep = "="*72; thin = "-"*72
    print(f"\n{sep}")
    print(f" REITSMA BIVARIATE RE - {biomarker_label}")
    print(f"{sep}")
```

Cosa fa: Definisce una funzione che stampa i risultati in modo simile a `summary(reitsma)` in R.

- `sep`: una linea di 72 caratteri =
- `f"...`: stringhe formattate (posso inserire variabili con `{nome_variabile}`)

```
hdr = f" {'Studio':<28} {'Se%':>5} {' [95%CI] ':<16} {'Sp%':>5} {' [95%CI] ':<16} {'N':>5}
print(hdr); print(f" {thin}")
```

Cosa fa: Stampa l'intestazione della tabella degli studi.

- `<28`: allinea a sinistra, 28 caratteri
- `>5`: allinea a destra, 5 caratteri

```
nSe = d['tp']+d['fn']; nSp = d['tn']+d['fp']
lo_se,hi_se = wilson(d['Se'],nSe)
lo_sp,hi_sp = wilson(d['Sp'],nSp)
```

Cosa fa: Calcola gli IC Wilson per ogni studio.

```
for i,s in enumerate(d['study']):
    sl = s.replace('\n',' ')
    print(f" {sl:<28} {d['Se'][i]*100:5.1f} [{lo_se[i]*100:.1f}-{hi_se[i]*100:.1f}] {
        f" {d['Sp'][i]*100:5.1f} [{lo_sp[i]*100:.1f}-{hi_sp[i]*100:.1f}] {'':<4}"
        f" {d['N'][i]:5} {d['CTp'][i]:4}")
```

Cosa fa: Stampa una riga per ogni studio con sensibilità, specificità, N e CT+.

```
print(f"\n POOLED ESTIMATES (random-effects bivariate):")
print(f"   Sensitivity {Se['est']*100:.1f}% 95%CI [{Se['lo']*100:.1f}% - {Se['hi']*100:.1f}%]
print(f"   Specificity {Sp['est']*100:.1f}% 95%CI [{Sp['lo']*100:.1f}% - {Sp['hi']*100:.1f}%]
```

Cosa fa: Stampa le stime pooled con intervalli di confidenza.

```
print(f"\n HETEROGENEITY:")
print(f"   Sensitivity I2={{Se['I2']:4.0f}}% 2={{Se['tau2']:4.0f}}% "
      f"Q={{Se['Q']:.2f}} (df={{Se['df']}}, p={{Se['pQ']:.4f}})")
print(f"   Specificity I2={{Sp['I2']:4.0f}}% 2={{Sp['tau2']:4.0f}}% "
      f"Q={{Sp['Q']:.2f}} (df={{Sp['df']}}, p={{Sp['pQ']:.4f}})")
```

Cosa fa: Stampa le misure di eterogeneità.

```
print(f"\n CLINICAL METRICS (prev CT+ = {{res['prev']*100:.1f}}%):")
print(f"   NPV {{res['npv']*100:.2f}}%")
print(f"   CT scans avoided {{res['ct']:.1f}}%")
print(f"   Missed lesions/1000 {{res['miss']:.1f}}%")
```

Cosa fa: Stampa le metriche cliniche (NPV, TAC evitate, lesioni mancate).

```
d2 = {k2:(v[:-1] if isinstance(v,np.ndarray) else v) for k2,v in d.items() if k2!='study'}
d2['study'] = d['study'][:-1]; d2 = enrich(d2)
```



```
sa_Se = DL(d2['lSe'],d2['vSe']); sa_Sp = DL(d2['lSp'],d2['vSp'])
```

Cosa fa: Sensitivity analysis — riesegue l'analisi escludendo l'ultimo studio.

- `v[:-1]`: tutti gli elementi tranne l'ultimo
- Questo verifica se i risultati dipendono da un singolo studio

```
last = d['study'][-1].replace('\n',' ')
print(f"\n SENSITIVITY ANALYSIS (excl. '{last}', k={res['k']-1}):")
print(f"    Se {sa_Se['est']*100:.1f}% [{sa_Se['lo']*100:.1f}-{sa_Se['hi']*100:.1f}]
      f"Δ=({ Se['est']-sa_Se['est'])*100:+.1f} pp")
print(f"    Sp {sa_Sp['est']*100:.1f}% [{sa_Sp['lo']*100:.1f}-{sa_Sp['hi']*100:.1f}]
      f"Δ=({ Sp['est']-sa_Sp['est'])*100:+.1f} pp")
```

Cosa fa: Stampa i risultati della sensitivity analysis con la differenza (Δ) in punti percentuali (pp).

PARTE 9: Forest Plot Publication Ready

```
def forest_plot_pub(res, col_main, col_light, fname, fig_label):
    d    = res['d']
    k    = res['k']
    Se   = res['Se']; Sp = res['Sp']
```

Cosa fa: Definisce la funzione per creare forest plot in stile rivista scientifica.

```
nSe = d['tp']+d['fn']; nSp = d['tn']+d['fp']
lo_se,hi_se = wilson(d['Se'],nSe)
lo_sp,hi_sp = wilson(d['Sp'],nSp)
```

Cosa fa: Calcola gli IC Wilson per ogni studio.

```
w_se = 1/d['vSe']; w_se_n = w_se/w_se.max()
w_sp = 1/d['vSp']; w_sp_n = w_sp/w_sp.max()
```

Cosa fa: Calcola i pesi per la dimensione dei quadrati.

- `w_se`: peso = $1/\text{varianza}$
 - `w_se_n`: peso normalizzato (tra 0 e 1) per la dimensione grafica
- ```
y = np.arange(k-1, -1, -1, dtype=float) # k,k-1,...,0
y_p = -1.2 # pooled row
```

**Cosa fa:** Definisce le posizioni verticali degli studi e del rombo pooled.

- `y`: [7, 6, 5, 4, 3, 2, 1, 0] per 8 studi (dall'alto in basso)
- `y_p` = -1.2: il rombo pooled sta sotto tutti gli studi

```

fig = plt.figure(figsize=(14, max(5, k*0.65+2.2)))
gs = gridspec.GridSpec(1, 2, figure=fig,
 left=0.01, right=0.99,
 wspace=0.06)
axes = [fig.add_subplot(gs[0]), fig.add_subplot(gs[1])]

```

**Cosa fa:** Crea la figura con 2 pannelli affiancati.

- `figsize=(14, ...)`: larghezza 14 pollici
- `GridSpec(1, 2)`: 1 riga, 2 colonne
- `left=0.01, right=0.99`: margini stretti (massimizza lo spazio)
- `wspace=0.06`: spazio minimo tra i due pannelli

```

for ax_idx, (ax, vals, lo, hi, pool_r, w_n, xlabel, ci_tag) in enumerate([
 (axes[0], d['Se'], lo_se, hi_se, Se, w_se_n, "Sensitivity (95% CI)", "Se"),
 (axes[1], d['Sp'], lo_sp, hi_sp, Sp, w_sp_n, "Specificity (95% CI)", "Sp"),
]):

```

**Cosa fa:** Itera sui due pannelli (sensibilità e specificità).

```

for i, yi in enumerate(y):
 if i % 2 == 0:
 ax.axhspan(yi-0.45, yi+0.45, color='#F7F7F7', zorder=0)

```

**Cosa fa:** Disegna le **bande alternate** grigie per leggere meglio le righe.

- `axhspan`: rettangolo orizzontale
- `i % 2 == 0`: righe pari (0, 2, 4, ...) hanno lo sfondo grigio
- `zorder=0`: sta dietro a tutto

```

ax.axhspan(y_p-0.45, y_p+0.45, color='#EEF4FF' if col_main==COL_GFAP else '#EEF9F1',
 zorder=0)

```

**Cosa fa:** Evidenzia la riga del pooled con un colore diverso (azzurro per GFAP, verde per S100B).

```

for i, yi in enumerate(y):
 ax.plot([lo[i], hi[i]], [yi, yi],
 color=col_main, lw=1.2, solid_capstyle='round', zorder=3)
 ax.plot([lo[i], lo[i]], [yi-0.1, yi+0.1],
 color=col_main, lw=1.2, zorder=3)
 ax.plot([hi[i], hi[i]], [yi-0.1, yi+0.1],
 color=col_main, lw=1.2, zorder=3)
 sz = 40 + w_n[i]*140
 ax.scatter(vals[i], yi, s=sz, color=col_main,
 zorder=4, edgecolors='white', linewidth=0.8)

```

**Cosa fa:** Disegna ogni studio:

1. **Linea dell'IC:** da `lo[i]` a `hi[i]` alla posizione `yi`
2. **Tacche alle estremità:** piccole linee verticali ai bordi dell'IC
3. **Quadrato:** posizione = stima puntuale, dimensione = peso dello studio
4. `solid_capstyle='round'`: estremità arrotondate (più elegante)

```
dw = (pool_r['hi'] - pool_r['lo']) / 2
est = pool_r['est']
diamond = Polygon(
 [[est-dw, y_p], [est, y_p+0.35], [est+dw, y_p], [est, y_p-0.35]],
 closed=True, facecolor=col_main, edgecolor='white', lw=1.2, zorder=5
)
ax.add_patch(diamond)
```

**Cosa fa:** Disegna il rombo pooled.

- I 4 vertici del rombo: centro = stima pooled, larghezza = ampiezza dell'IC
- `Polygon`: crea la forma geometrica
- `ax.add_patch()`: la aggiunge al grafico

```
for i, yi in enumerate(y):
 ax.text(1.04, yi, f'{vals[i]*100:.1f}',
 va='center', ha='left', fontsize=7.5, color=COL_GREY,
 transform=ax.get_yaxis_transform())
```

**Cosa fa:** Aggiunge i valori numerici a destra di ogni studio.

- 1.04: posizione sull'asse X (fuori dal grafico, a destra)
- `transform=ax.get_yaxis_transform()`: le coordinate X sono relative all'asse (0-1), Y sono in coordinate dati

```
ax.text(0.98, -0.02,
 f"I2= {pool_r['I2']:.0f}% 2= {pool_r['tau2']:.3f} "
 f"Q= {pool_r['Q']:.1f} (p= {pool_r['pQ']:.3f})",
 transform=ax.transAxes, ha='right', va='bottom',
 fontsize=7, color='#666666', style='italic')
```

**Cosa fa:** Aggiunge l'annotazione dell'eterogeneità in basso a destra.

- `transform=ax.transAxes`: coordinate relative all'asse (0-1)
- 0.98, -0.02: angolo in basso a destra, leggermente sotto l'asse

```
ax.spines['left'].set_visible(False)
ax.tick_params(left=False)
```

**Cosa fa:** Rimuove l'asse verticale sinistro (più pulito, stile rivista).

```
se_str = f"Se= {Se['est']*100:.1f}% [{Se['lo']*100:.1f}-{Se['hi']*100:.1f}]"
sp_str = f"Sp= {Sp['est']*100:.1f}% [{Sp['lo']*100:.1f}-{Sp['hi']*100:.1f}]"
fig.suptitle(
```

```

f"Figure {fig_label}. Forest plot - {res['label']}"
f"({k} studies, N={res['N']:,})\n"
f"Pooled: {se_str} {sp_str}",
fontsize=10, fontweight='bold', y=1.01, x=0.5, ha='center'
)

```

**Cosa fa:** Aggiunge il **titolo della figura** in alto, centrato.

- `y=1.01`: leggermente sopra il grafico
- `x=0.5, ha='center'`: centrato orizzontalmente

```

fig.text(0.5, -0.01,
 "Squares proportional to study weight (1/variance). "
 "Diamond = pooled estimate (DerSimonian-Laird bivariate RE). "
 "Vertical dashed line = pooled estimate.",
 ha='center', fontsize=7.5, color='#555555', style='italic')

```

**Cosa fa:** Aggiunge la **nota esplicativa** in basso (come richiedono le riviste scientifiche).

```

plt.savefig(fname, dpi=300, bbox_inches='tight',
 facecolor='white', edgecolor='none')
plt.close()

```

**Cosa fa:** Salva la figura come file PNG.

- `dpi=300`: alta risoluzione (qualità stampa)
- `bbox_inches='tight'`: taglia i margini inutili
- `facecolor='white'`: sfondo bianco

## PARTE 10: Curva SROC

```

def sroc_points(d):
 D = d['lSe'] + d['lSp']; S = d['lSe'] - d['lSp']
 w = 1 / (d['vSe'] + d['vSp'])
 mS = np.sum(w*S) / np.sum(w); mD = np.sum(w*D) / np.sum(w)
 b = np.sum(w*(S-mS)*(D-mD)) / np.sum(w*(S-mS)**2)
 a = mD - b*mS
 t = np.linspace(-4.5, 4.5, 400)
 return 1 - expit(-(a-t)/2), expit((a+t)/2)

```

**Cosa fa:** Genera i punti della curva SROC con il metodo **Moses-Shapiro-Littenberg**.

**Spiegazione passo passo:**

1.  $D = \text{logit}(\text{Se}) + \text{logit}(\text{Sp})$ : la “distanza diagnostica” totale
2.  $S = \text{logit}(\text{Se}) - \text{logit}(\text{Sp})$ : la “soglia” del test
3. **Regressione pesata  $D \sim S$** : trova la relazione lineare tra D e S

4. **Genera la curva:** per ogni valore di  $t$  (da  $-4.5$  a  $+4.5$ ), calcola  $Se$  e  $Sp$  predette

**Cosa mostra la curva SROC:** - Asse X:  $1 - \text{Specificità}$  (falsi positivi) - Asse Y: Sensibilità (veri positivi) - Una curva più vicina all'angolo in alto a sinistra = test migliore

```
fig4, ax4 = plt.subplots(figsize=(7, 7))
```

**Cosa fa:** Crea una figura quadrata  $7 \times 7$  pollici.

```
fpr_g, tpr_g = sroc_points(G['d'])
fpr_s, tpr_s = sroc_points(S['d'])
ax4.plot(fpr_g, tpr_g, color=COL_GFAP, lw=2.2, label='GFAP+UCH-L1', zorder=4)
ax4.plot(fpr_s, tpr_s, color=COL_S100B, lw=2.2, label='S100B', zorder=4)
```

**Cosa fa:** Disegna le due curve SROC sovrapposte.

```
max_N = max(G['d']['N'].max(), S['d']['N'].max())
for d_, col, col_l in [(G['d'], COL_GFAP, COL_GFAP_L), (S['d'], COL_S100B, COL_S100B_L)]:
 sizes = (d_['N']/max_N)*350
 ax4.scatter(1-d_['Sp'], d_['Se'], s=sizes,
 facecolor=col_l, edgecolors=col, lw=1.2, zorder=3, alpha=0.85)
```

**Cosa fa:** Aggiunge i punti dei singoli studi, con area proporzionale a  $N$ .

```
ax4.scatter(1-G['Sp']['est'], G['Se']['est'],
 marker='D', s=180, facecolor=COL_GFAP,
 edgecolors='white', lw=1.8, zorder=6)
ax4.scatter(1-S['Sp']['est'], S['Se']['est'],
 marker='s', s=180, facecolor=COL_S100B,
 edgecolors='white', lw=1.8, zorder=6)
```

**Cosa fa:** Aggiunge i punti sommario (rombo per GFAP, quadrato per S100B).

```
for res_, col in [(G, COL_GFAP), (S, COL_S100B)]:
 Se_res_ = res_['Se']; Sp_res_ = res_['Sp']
 n_pts = 80
 theta = np.linspace(0, 2*np.pi, n_pts)
 hw_se = (Se_res_['hi'] - Se_res_['lo'])/2
 hw_sp = (Sp_res_['hi'] - Sp_res_['lo'])/2
 ell_x = (1 - Sp_res_['est']) + hw_sp*np.cos(theta)
 ell_y = Se_res_['est'] + hw_se*np.sin(theta)
 ax4.plot(ell_x, ell_y, color=col, lw=0.9, ls='--', alpha=0.5, zorder=3)
```

**Cosa fa:** Disegna le **ellissi di confidenza** approssimate attorno ai punti sommario.

- Usa l'equazione parametrica dell'ellisse:  $x = \text{centro}_x + a \times \cos()$ ,  $y = \text{centro}_y + b \times \sin()$
- $hw\_se$  e  $hw\_sp$  sono i semi-assi dell'ellisse (metà ampiezza dell'IC)

```
ax4.plot([0,1],[0,1], color='#AAAAAA', lw=0.8, ls=':', zorder=1)
```

**Cosa fa:** Disegna la **linea di caso** (diagonale). Un test che cade su questa linea non è meglio del lanciare una moneta.

---

## PARTE 11: Clinical Summary Panel (4 Pannelli)

```
fig5 = plt.figure(figsize=(14, 11))
gs5 = gridspec.GridSpec(2, 2, figure=fig5,
 hspace=0.42, wspace=0.36,
 left=0.08, right=0.97,
 top=0.92, bottom=0.08)
```

**Cosa fa:** Crea una figura con 4 pannelli disposti in griglia 2×2.

- `hspace=0.42`: spazio verticale tra le righe
- `wspace=0.36`: spazio orizzontale tra le colonne

### Pannello A: Sensibilità e Specificità

```
ax_a = fig5.add_subplot(gs5[0,0])
x = np.array([0, 1.1])
w = 0.45
se_v = np.array([G['Se']['est'], S['Se']['est']])*100
sp_v = np.array([G['Sp']['est'], S['Sp']['est']])*100
```

**Cosa fa:** Prepara i dati per il grafico a barre di Se e Sp.

```
bars_se = ax_a.bar(x-w/2, se_v, w, color=cols, alpha=0.88,
 edgecolor='white', lw=1.2, label='Sensitivity')
bars_sp = ax_a.bar(x+w/2, sp_v, w, color=cols_1, alpha=0.88,
 edgecolor=[COL_GFAP, COL_S100B], lw=1.2,
 hatch='//', label='Specificity')
```

**Cosa fa:** Disegna le barre affiancate.

- `x-w/2`: barre della sensibilità a sinistra
- `x+w/2`: barre della specificità a destra
- `hatch='//'`: tratteggio sulle barre della specificità (per distinguerle anche in bianco e nero)

```
ax_a.errorbar(x-w/2, se_v, yerr=[se_v-se_lo, se_hi-se_v],
 fmt='none', color='#333333', capsize=5, lw=1.3, capthick=1.3)
```

**Cosa fa:** Aggiunge le **barre di errore** (intervalli di confidenza).

- `yerr=[se_v-se_lo, se_hi-se_v]`: errori asimmetrici (l'IC non è sempre simmetrico)

```
ax_a.axhline(95, color='#CC4444', lw=0.8, ls='--', alpha=0.5)
ax_a.text(1.6, 95.8, '95%', fontsize=7.5, color='#CC4444')
```

**Cosa fa:** Aggiunge una **linea di riferimento a 95%** (soglia clinica importante).

### Pannello B: NPV

```
ax_b = fig5.add_subplot(gs5[0,1])
npv_v = np.array([G['npv'], S['npv']])*100
b_npv = ax_b.bar(x, npv_v, 0.55, color=cols, alpha=0.88,
 edgecolor='white', lw=1.2)
ax_b.set_ylim(97, 100.3)
```

**Cosa fa:** Grafico a barre del NPV.

- `set_ylim(97, 100.3)`: l'asse Y parte da 97% (non da 0) per evidenziare le piccole differenze
- **Attenzione:** questo “zoom” può essere fuorviante se non comunicato chiaramente

### Pannello C: TAC Evitate e Lesioni Manca

```
ax_c = fig5.add_subplot(gs5[1,0])
ct_v = np.array([G['ct'], S['ct']])
miss_v = np.array([G['miss'], S['miss']])
xc = np.array([0, 0.9])
bars_ct = ax_c.bar(xc-0.22, ct_v, 0.4, color=cols, alpha=0.88,
 edgecolor='white', lw=1.2, label='CT avoiding (%)')
bars_miss = ax_c.bar(xc+0.22, miss_v, 0.4, color=cols_l, alpha=0.88,
 edgecolor=[COL_GFAP,COL_S100B], lw=1.2,
 hatch='//', label='Missed lesions/1000 pts')
```

**Cosa fa:** Grafico a barre comparativo per TAC evitate e lesioni mancate.

- Due gruppi di barre per ogni biomarcatore
- Le barre con tratteggio rappresentano le lesioni mancate

### Pannello D: Tabella Riassuntiva

```
ax_d = fig5.add_subplot(gs5[1,1])
ax_d.axis('off')
```

**Cosa fa:** Crea un pannello vuoto (senza assi) per la tabella.

```
table_data = [
 ['Parameter', 'GFAP+UCH-L1', 'S100B'],
 ['Studies (k)', '8', '6'],
 ['Patients (N)', '4,550', '3,582'],
```

```
 ...
]
```

**Cosa fa:** Definisce i dati della tabella come lista di liste.

```
tbl = ax_d.table(
 cellText = [r[1:] for r in table_data[1:]],
 rowLabels = [r[0] for r in table_data[1:]],
 colLabels = table_data[0][1:],
 loc = 'center',
 cellLoc = 'center'
)
tbl.auto_set_font_size(False)
tbl.set_fontsize(8.5)
tbl.scale(1.05, 1.75)
```

**Cosa fa:** Crea la tabella e ne configura l'aspetto.

- `tbl.scale(1.05, 1.75)`: allarga le celle ( $1.05\times$  in larghezza,  $1.75\times$  in altezza)

```
for (ri, ci), cell in tbl.get_celld().items():
 cell.set_edgecolor('#CCCCCC')
 cell.set_linewidth(0.6)
 if ri == 0:
 cell.set_facecolor(COL_GFAP if ci==0 else COL_S100B)
 cell.get_text().set_color('white')
 cell.get_text().set_fontweight('bold')
 elif ri % 2 == 0:
 cell.set_facecolor('#F4F4F4')
```

**Cosa fa:** Personalizza l'aspetto di ogni cella.

- Intestazione: sfondo colorato con testo bianco
- Righe alternate: sfondo grigio chiaro per leggibilità

---

## PARTE 12: Table 1 — Tabella Publication Ready

```
fig_t, ax_t = plt.subplots(figsize=(13, 8))
ax_t.axis('off')

col_hdr = ['Study\n(Reference)', 'Design', 'N', 'CT+\n(%)', 'Cutoff\n(µg/L)',
 'Window', 'TP', 'FP', 'FN', 'TN',
 'Sensitivity\n(95% CI)', 'Specificity\n(95% CI)']
```

**Cosa fa:** Crea una figura per la **tabella completa degli studi** (Table 1), come quelle che si trovano negli articoli scientifici.



- 12 colonne con tutte le informazioni rilevanti
- \n per andare a capo nelle intestazioni

```
rows_g = []
for i,s in enumerate(GFAP_RAW['study']):
 d_ = GFAP_RAW
 N_ = d_['TP'][i]+d_['FP'][i]+d_['FN'][i]+d_['TN'][i]
 CTp= d_['TP'][i]+d_['FN'][i]
 tp_=float(d_['TP'][i])+0.5*(d_['TP'][i]==0)
 ...
 rows_g.append(...)
```

**Cosa fa:** Costruisce le righe della tabella per ogni studio GFAP.

- $0.5*(d\_['TP'][i]==0)$ : aggiunge 0.5 solo se il valore è 0 (correzione di continuità)

```
all_rows = (rows_g + [pooled_row(G,'GFAP+UCH-L1','FDA-cleared',' 12 h')] +
 rows_s + [pooled_row(S,'S100B','0.10 µg/L',' 6 h')])
```

**Cosa fa:** Combina tutte le righe: studi GFAP + riga pooled GFAP + studi S100B + riga pooled S100B.

```
tbl_t = ax_t.table(
 cellText = all_rows,
 colLabels = col_hdr,
 loc = 'center',
 cellLoc = 'center'
)
tbl_t.auto_set_font_size(False)
tbl_t.set_fontsize(7.2)
tbl_t.scale(1, 2.2)
```

**Cosa fa:** Crea la tabella con tutte le righe.

- fontsize=7.2: carattere piccolo per stare tutto in una pagina
- scale(1, 2.2): celle molto alte (2.2×) per leggibilità

```
n_g = len(rows_g); n_s = len(rows_s)
for (ri,ci), cell in tbl_t.get_celld().items():
 cell.set_edgecolor('#BBBBBB'); cell.set_linewidth(0.5)
 if ri == 0:
 cell.set_facecolor('#1A1A2E')
 cell.get_text().set_color('white')
 cell.get_text().set_fontweight('bold')
 elif 1 <= ri <= n_g:
 cell.set_facecolor('#EEF3FA' if ri%2==1 else '#DDEAF6')
 elif ri == n_g+1:
 cell.set_facecolor(COL_GFAP)
```

```

cell.get_text().set_color('white')
cell.get_text().set_fontweight('bold')
...

```

**Cosa fa:** Colora le celle in modo differenziato:

| Sezione      | Colore                             |
|--------------|------------------------------------|
| Intestazione | Blu scuro quasi nero, testo bianco |
| Studi GFAP   | Azzurro alternato chiaro/scuro     |
| Pooled GFAP  | Blu scuro, testo bianco            |
| Studi S100B  | Verde alternato chiaro/scuro       |
| Pooled S100B | Verde scuro, testo bianco          |

## 8. Perché Python per le Figure “Publication Ready”

### Il Problema con R

R (con i pacchetti standard) produce grafici **funzionali ma non belli**:

- I forest plot di `meta::forest()` sono basilari
- Personalizzare ogni dettaglio è complicato
- Le tabelle sono difficili da esportare come immagini

### I Vantaggi di Python per le Figure

| Aspetto                 | R          | Python (matplotlib) |
|-------------------------|------------|---------------------|
| Controllo su ogni pixel | Limitato   | Totale              |
| Stile personalizzabile  | Medio      | Altissimo           |
| Tabelle come immagini   | Difficile  | Naturale            |
| Layout multi-pannello   | Macchinoso | Elegante (GridSpec) |
| Riproducibilità         | Buona      | Ottima              |

### Cosa Significa “Publication Ready”

Una figura publication-ready:

1. **Risoluzione alta** (300 DPI) → si stampa bene
2. **Font corretti** → leggibile anche ridotta
3. **Colori accessibili** → distinguibili anche in bianco e nero e per daltonici
4. **Note esplicative** → il lettore capisce senza leggere il testo
5. **Allineamento perfetto** → tutto è allineato e proporzionato
6. **Niente elementi superflui** → nessuna linea, nessun testo inutile

Python con matplotlib permette di controllare **ogni singolo pixel** della figura.

## 9. Riassunto Finale

### Cosa Abbiamo Imparato

| Concetto                                | Spiegazione Breve                                                           |
|-----------------------------------------|-----------------------------------------------------------------------------|
| <b>Sensibilità</b>                      | Quanto il test è bravo a trovare chi ha la malattia                         |
| <b>Specificità</b>                      | Quanto il test è bravo a escludere chi non ce l'ha                          |
| <b>NPV</b>                              | Se il test è negativo, quanto mi posso fidare                               |
| <b>Meta-analisi</b>                     | Combinare studi piccoli per ottenere un risultato grande                    |
| <b>DerSimonian-Laird</b>                | Un metodo per fare la media pesata tenendo conto che gli studi sono diversi |
| <b><math>\tau^2</math> (tau quadro)</b> | Quanto gli studi differiscono tra loro (eterogeneità)                       |
| <b><math>I^2</math></b>                 | Percentuale di differenza dovuta a cause vere, non al caso                  |
| <b>Forest plot</b>                      | Grafico che mostra tutti gli studi e il risultato combinato                 |
| <b>SROC</b>                             | Curva che riassume quanto è bravo il test                                   |
| <b>Sensitivity analysis</b>             | Verificare se il risultato cambia togliendo un studio                       |

### Le Differenze R vs Python

|                           | R (v9)                     | Python (v10)                           |
|---------------------------|----------------------------|----------------------------------------|
| <b>Modello</b>            | Reitsma bivariato completo | DerSimonian-Laird marginale            |
| <b>Correlazione Se-Sp</b> | Sì                         | No                                     |
| <b>Risultati</b>          | Leggermente diversi        | Leggermente diversi                    |
| <b>Bias</b>               | Nessuno                    | Nessuno grave, ma modello più semplice |
| <b>Figure</b>             | Basic                      | Publication-ready                      |

## Python Introduce Bias?

**No, non in modo significativo.** Le differenze con R sono dovute al modello statistico (più semplice), non a errori di programmazione. Per una pubblicazione:

- Python va benissimo per le figure e le tabelle
- Per le stime pooled, è meglio usare R/**mada** (modello più completo)
- La strategia migliore: **entrambi** — R per le stime, Python per le figure

## Il Messaggio Clinico

**Entrambi i biomarcatori (GFAP+UCH-L1 e S100B) hanno un NPV eccellente (>98%), il che significa che un test negativo ci dà grande sicurezza che il paziente non abbia lesioni intracraniche. Questo potrebbe permettere di evitare molte TAC inutili, riducendo radiazioni e costi.**

---

*Documento creato come materiale didattico per studenti delle scuole superiori. Tutti i concetti statistici sono spiegati con analogie ed esempi pratici.*